

HL7 validation: the three tiers

Early Access · MessageFoundry v0.3.2 · September 2026

MessageFoundry validates HL7 v2.x at three tiers. Each catches a different kind of problem. Choose the tiers each feed needs. They can work together. Tolerant parsing handles non-conformant messages, while strict validation can reject off-spec input.

Tier	Engine	When	Checks	On failure
1. Tolerant peek	<code>python-hl7</code> (<code>parsing/peek.py</code>)	Always (hot path)	Parse + fast field access. Size/segment caps. MSH present	Unparseable / oversized → ERROR (NAK), never crashes the connection
2. Strict structural	<code>hl7apy</code> (<code>parsing/validate.py</code>)	Opt-in per inbound (<code>strict=True</code>)	Version-aware schema : segment cardinality, datatypes, table values, lengths, MSH-12 version	Non-conformant → synchronous NAK (AR/AE) at the listener
3. Business consistency	<code>parsing/consistency.py</code> (this WP)	In a Router/Handler	Cross-field coherence the schema cannot express	Handler decides: FILTERED or ERROR /dead-letter

Tier 1 — tolerant peek (always on)

The hot path uses `python-hl7` for tolerant field access, such as `msg["PID-3"]`. Routing does not run full structural validation. It checks message-size and segment limits, and requires MSH. Unparseable or oversized messages enter the error/dead-letter path with **ERROR** logged. They do not crash the connection. This default preserves the raw message and makes failures visible to operators.

Tier 2 — strict structural validation (opt-in)

Enable `hl7apy` strict validation on the **inbound** connection when off-spec messages must be rejected before routing. It checks the official HL7 structure for the message version. This slower path is **opt-in per connection** and stays outside routing:

```
from messagefoundry import MLLP, inbound

inbound("IB_ACME_ADT", MLLP(port=2575), router="adt_router", strict=True, hl7_version="2.5")
```

The listener **NAKs a non-conformant message synchronously** (AR/AE), before routing. Set `hl7_version` explicitly. Do not rely on silent autodetection. Enable strict validation when a downstream system rejects malformed structure, the feed promises conformance, or correctness outweighs throughput. Leave it off for tolerant archival or forwarding feeds.

Strict validation surfaces a single conformance error (not a full report) — a full report of a PHI message would be a data leak. See `parsing/validate.py`.

Tier 3 — cross-field business consistency (Router/Handler)

Strict validation checks items against the schema **independently**. It does not check related values together: required identifiers, values repeated across segments, or whether admit ≤ discharge. Under **ASVS 2.2.3 / 2.1.2**, the application must check these relationships. In MessageFoundry, the **Router/Handler** does that work.

`messagefoundry/parsing/consistency.py` provides small, **generic, composable** primitives. Each takes the parsed `Message` plus field *paths* and returns a list of **PHI-safe Violations** (rule + path, **never the field value**):

Primitive	Checks
<code>required(msg, *paths)</code>	each path is present (non-empty)
<code>same_across(msg, *paths)</code>	the paths all hold the same value (catches differing values <i>and</i> present-vs-absent)
<code>valid_date(msg, path)</code>	the value is a well-formed HL7 date/time (when present)
<code>dates_in_order(msg, earlier, later)</code>	<code>earlier ≤ later</code> (when both present & valid)
<code>matches(msg, path, pattern)</code>	the value fully matches a regex (a field-format allow-list)
<code>check(*groups)</code>	flattens several results into one list for a single decision

The library **detects. The Handler decides**. Keeping it pure preserves the at-least-once *re-run* invariant (a Handler must be a pure function of the message — CLAUDE.md §2). Compose the generic primitives into your feed's message-type-specific rules:

```
from messagefoundry import Send, handler
from messagefoundry.parsing.consistency import (
    ConsistencyError, check, dates_in_order, required, same_across, valid_date,
)

@handler("validate_and_archive")
def validate_and_archive(msg):
    violations = check(
        required(msg, "PID-3", "PID-5", "MSH-10"), # patient id, name, control id present
        same_across(msg, "MSH-9.2", "EVN-1"),      # trigger event echoed in EVN-1
        valid_date(msg, "PID-7"),                   # date of birth well-formed
        dates_in_order(msg, "PV1-44", "PV1-45"),    # admit ≤ discharge
    )
    if violations:
```

```
raise ConsistencyError(violations)          # → ERROR / dead-letter (operator sees it)
# ...or `return None` to drop it silently (logged FILTERED)
return Send("FILE-OUT_ACME_ADT", msg)
```

Acting on a non-empty result is a deliberate choice:

- `raise ConsistencyError(violations)` → the message goes to the error/dead-letter path (logged `ERROR`, surfaced via the AlertSink). Use this when a downstream system cannot safely consume it. The exception's string is built from rule + paths only, so it is **safe to log and store** (no PHI).
- `return None` → the message is dropped and logged `FILTERED`. Use this for a benign coherence issue you would rather not forward.

A full worked example — wired inbound → router → handler → file — is in

`samples/consistency/validated_adt.py`:

```
python -m messagefoundry serve --config samples/consistency --db ./mf.db --env dev
```

PHI-safety

A `Violation` records the **rule and field paths**, never field values. It can be logged, aggregated, or included in a stored disposition through `safe_exc()`. See `PHI.md` §7. A date violation reads *"PID-7 is not a valid HL7 date/time"* without including the patient value.

Choosing tiers

- **Most feeds:** Tier 1 only — tolerant, route bad messages to `ERROR`.
- **Add Tier 2** (`strict=True`) when off-spec *structure* must be rejected at the door.
- **Add Tier 3** when *business rules across fields* matter (required identifiers, sane/ordered dates, cross-segment coherence) — independent of whether Tier 2 is on.